

↳ The longer these two processes compete, the more likely they are to achieve the desired percentage.

Ticket Mechanisms

1. Ticket Currency

↳ Currency allows users with a set of tickets to allocate tickets among their own jobs

The system then automatically converts said currency into the correct global value. (200 Total global tickets)

Given 100 global tickets

↑ User A → 500 (A's currency) to A1 → 50 (global currency)
↳ out of 1000 → 500 (A's currency) to A2 → 50 (global currency)

Given 100 global tickets ← User B → 10 (B's currency) to B1 → 100 (global currency)
↳ out of 10

2. Ticket transfer

↳ A process can temporarily hand off its tickets to another process.

↳ This is useful in a client/server setting, where a client process sends a message to a server asking it to do some work on its behalf

To speed up the work, the client can pass the tickets to the server & thus maximize the performance of the server.

When finished, the server sends back the client's tickets.

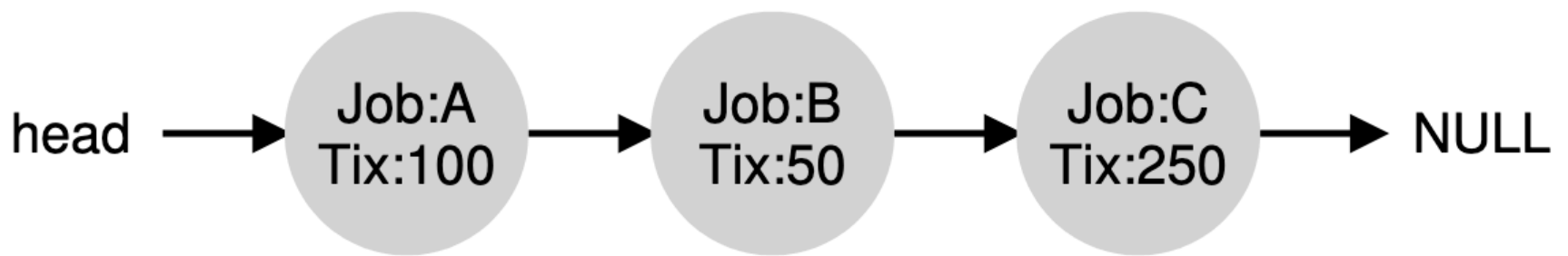
3. Ticket Inflation:

↳ A process can temporarily raise or lower the no. of tickets it owns.

↳ This is applied in an environment where a group of processes trust one another.

Implementation

§ All we need is a good random number generator & a data structure to track processes of the system.



§ To make a scheduling decision, we generate a random number (the winner) from the total number of tickets.

We then traverse the process list, adding each ticket value to the counter until it exceeds the winner.

Once this happens, the current list element is the winner.

In our example,

counter = 0 → counter = 100 (> 300?)

↓

(> 300?) counter = 400 ← counter = 150 (> 300?)
↪ C is winner

```
1 // counter: used to track if we've found the winner yet
2 int counter = 0;
3
4 // winner: call some random number generator to
5 //           get a value >= 0 and <= (totaltickets - 1)
6 int winner = getRandom(0, totaltickets);
7
8 // current: use this to walk through the list of jobs
9 node_t *current = head;
10 while (current) {
11     counter = counter + current->tickets;
12     if (counter > winner)
13         break; // found the winner
14     current = current->next;
15 }
16 // 'current' is the winner: schedule it...
```

An Example,

- ↳ We compare completion time of two jobs competing against one another, each with the same no. of tickets (100) & same runtime.
- ↳ In this scenario, we want each job to finish at roughly the same time but due to randomness of lottery scheduling one can finish before another.
- ↳ Fairness metric F , the time the first job completes divided by the time that the second job completes.

If $R=10$, first job finishes at 10 & second at 20 then

↓
runtime

$$F = \frac{10}{20}$$

$$= 0.5$$

↳ When both jobs finish at the same time, $F \approx 1$

↳ When job length is short, F is low.

Only when jobs run for a significant number of time slices does $F \approx 1$.

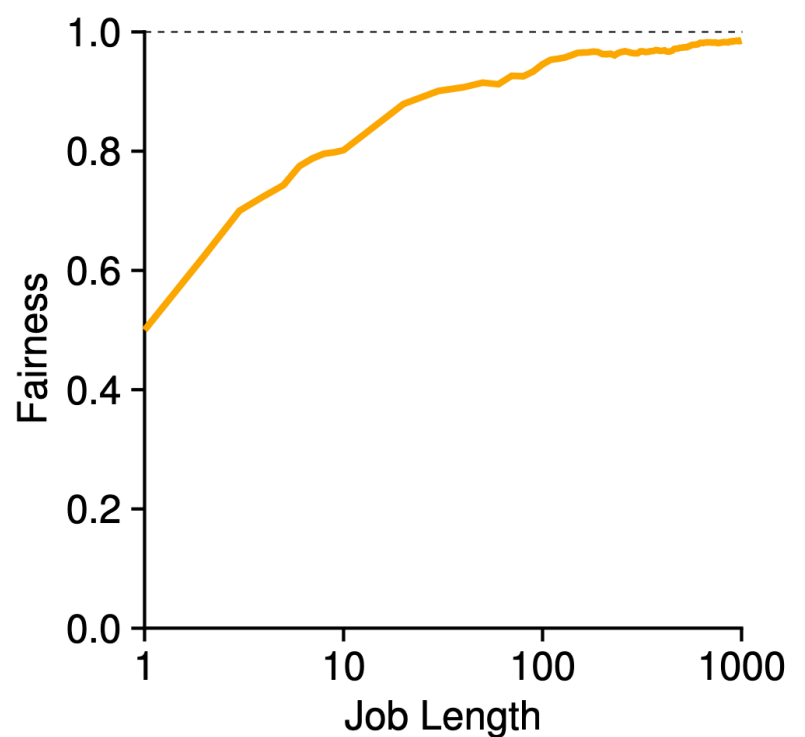


Figure 9.2: Lottery Fairness Study

Stride Scheduling

Each job has a stride, which is inversely proportional to number of tickets it has.

For Jobs A (100 tickets), B (50 tickets) & C (250 tickets),

we can compute the stride of each by dividing some large numbers by the number of tickets each process has.

So,

$$\text{Job A} = \frac{10,000}{100}$$

$$\text{Job B} = \frac{10,000}{50}$$

$$\text{Job C} = \frac{10,000}{250}$$

$$= 100$$

$$= 200$$

$$= 40$$

These values are strides of each process. Every time a process runs, we increment its counter (pass value) by its stride.

Hence, At any given time, pick the process that has the lowest stride.

```
curr = remove_min(queue); // pick client with min pass
schedule(curr);           // run for quantum
curr->pass += curr->stride; // update pass using stride
insert(queue, curr);      // return curr to queue
```

Pass(A) (stride=100)	Pass(B) (stride=200)	Pass(C) (stride=40)	Who Runs?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	...

Figure 9.3: Stride Scheduling: A Trace

Now, A ran twice, C five times & B once which proportional to their ticket values

But,

If a new job enters, what should be its pass value?

Should it be 0? if so, then it will monopolize the CPU.

The Linux Completely Fair Scheduler (CFS)

↳ It fairly divides a CPU evenly among all competing processes.

It does so through a simple counting based technique known as virtual runtime (vruntime).

↳ As each process runs, it accumulates vruntime. Each process's vruntime increases at the same rate.

The scheduler picks the process with the lowest vruntime.

↳ If CFS switches too often, fairness is increased but at the cost of performance (too much context switching).

If it switches less often, performance is increased (reduced context switching) but at the cost of fairness.

↳ CFS uses sched-latency to determine how long one process should run before considering a switch.

$$\text{time-slice} = \frac{\text{sched-latency}}{\text{no. of process (n)}}$$

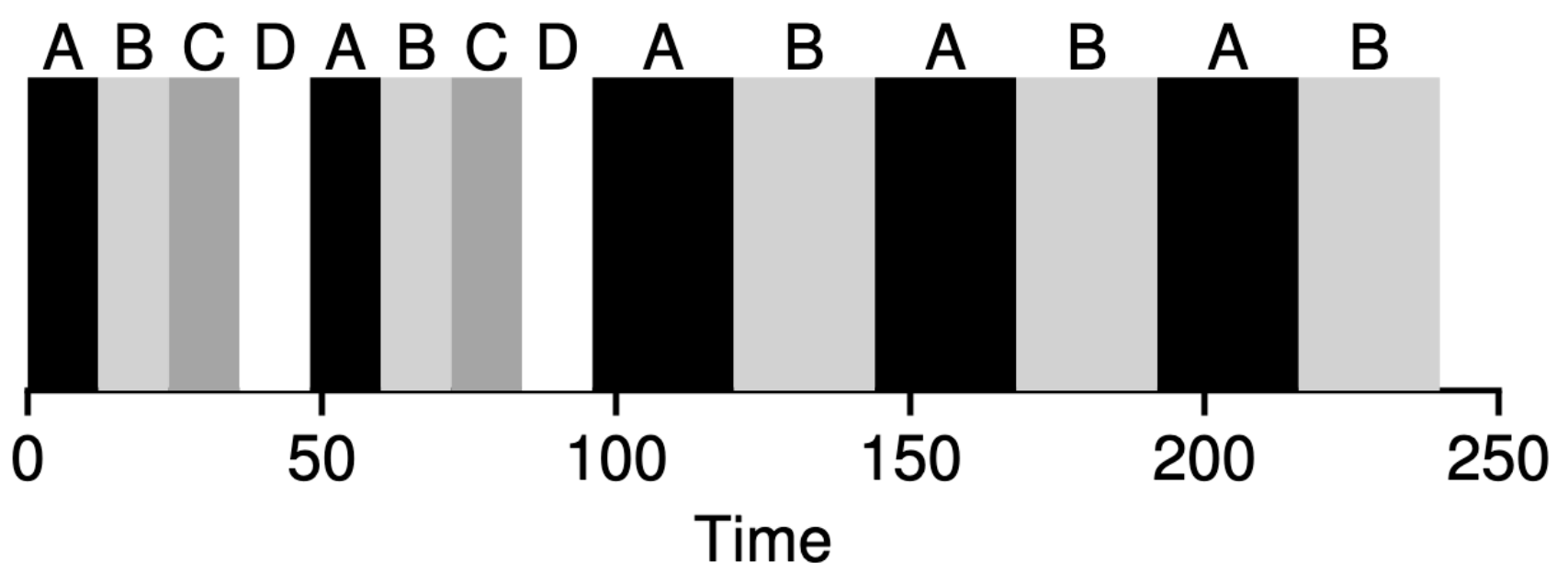


Figure 9.4: CFS Simple Example

↳ To ensure time slice doesn't become too small, we set a min value called min-granularity.

Weighting (Niceness)

↳ CFS enables control over process priority enabling users or admins to give some processes a higher share of CPU.

↳ It does so not with tickets but rather $nice \in [-20, +19]$ level of a process.

$nice > 0 \rightarrow$ lower priority

$nice < 0 \rightarrow$ higher priority

↳ CFS maps each nice value to a weight

```
static const int prio_to_weight[40] = {  
    /* -20 */ 88761, 71755, 56483, 46273, 36291,  
    /* -15 */ 29154, 23254, 18705, 14949, 11916,  
    /* -10 */ 9548, 7620, 6100, 4904, 3906,  
    /* -5 */ 3121, 2501, 1991, 1586, 1277,  
    /* 0 */ 1024, 820, 655, 526, 423,  
    /* 5 */ 335, 272, 215, 172, 137,  
    /* 10 */ 110, 87, 70, 56, 45,  
    /* 15 */ 36, 29, 23, 18, 15,  
};
```

↳ With these, we can compute the effective time slice as, with n processes

$$\text{time_slice}_k = \frac{\text{weight}_k}{\sum_{i=0}^{n-1} \text{weight}_i} \cdot \text{sched_latency}$$

Example,

We have Job A & B. A is given a nice value of -5 & B is default at 0.

So,

$$\text{weight}_A = 3121 \quad \& \quad \text{weight}_B = 1024$$

$$\begin{aligned} \text{time_slice}_A &= \frac{3121}{3121 + 1024} & \text{time_slice}_B &= \frac{1024}{3121 + 1024} \\ &= \frac{3}{4} \times 48 & &= \frac{1}{4} \times 48 \\ &= 36 & &= 12 \end{aligned}$$

↳ Now, vruntime becomes,

$$\text{vruntime}_i = \text{vruntime}_i + \frac{\text{weight}_0}{\text{weight}_i} \cdot \text{runtime}_i$$

$\downarrow$
 process's runtime
 across switches

$\downarrow$
 process i's
 accumulated
 runtime per
 switch

Red black Tree:

↳ When a scheduler has to find the next job to run, it should do so as soon as possible.

↳ Searching, inserting & Deletion should be fast

Data Structure	Search	Insert	Delete
Linked list	$O(n)$	$O(n)$	$O(n)$
↳ Sorted	$O(1)$	$O(n)$	$O(n)$
Binary Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$
↳ Worst	$O(n)$	$O(n)$	$O(n)$
Self balancing Trees	$O(\log n)$	$O(\log n)$	$O(\log n)$

↳ Red black tree

↳ CFS keeps only running & runnable processes in structure.

↳ Processes are ordered by vruntime & lowest one is chosen to run next.

Dealing with I/O And Sleeping Processes

↳ One problem with picking the lowest vruntime to run next arises with jobs that go to sleep for a long time.

↳ If Process A runs continuously & Process B has gone to sleep for 10 seconds.

When Process B will wake up then it will monopolize the CPU.

↳ To address, CFS assigns the lowest vruntime in the tree to the process B.

↳ Now, jobs that go to sleep frequently for short amount of time will never get their fair share of CPU.